

RxJS Observables

Reactive library for JavaScript

Prepared by:

TARUNA SHARMA

What is RxJS

RxJS is a javascript library that brings the concept of "reactive programming" to the web.

Reactive programming is just a different way of building software applications. Essentially, your software is built to "react" to changes that happen (like click events, data being fetched, etc) .

Normally you register event listeners.

```
document.addEventListener('click', () => console.log('Clicked!'));
```

Using RxJS you create an observable instead.

```
import { fromEvent } from 'rxjs';
```

```
fromEvent(document, 'click').subscribe(() => console.log('Clicked!'));
```

What is Reactive Programming??

Programming paradigm that works with asynchronous data streams. Ex:-

- UI Events
- Http Requests

A Stream?

A sequence of ongoing events ordered in time. Emits a **value**, **error** and complete **signal**.

Why do we need Reactive Programming?

The simple answer is we want to improve the user experience.

To keep the main thread free we need to do a lot of heavy and time-consuming work we want to do in the background. We also want to do heavy work and complex calculations on our servers as mobile devices are not very powerful to do the heavy lifting. So we need asynchronous work for network operations.

Observables are able to deliver values asynchronously.

Reactive programming



Pull vs Push

Pull and Push are two different protocols that describe how a data Producer can communicate with a data Consumer.

What is Pull? In Pull systems, the Consumer determines when it receives data from the data Producer. The Producer itself is unaware of when the data will be delivered to the Consumer.

Every JavaScript Function is a Pull system. The function is a Producer of data, and the code that calls the function is consuming it by "pulling" out a single return value from its call.

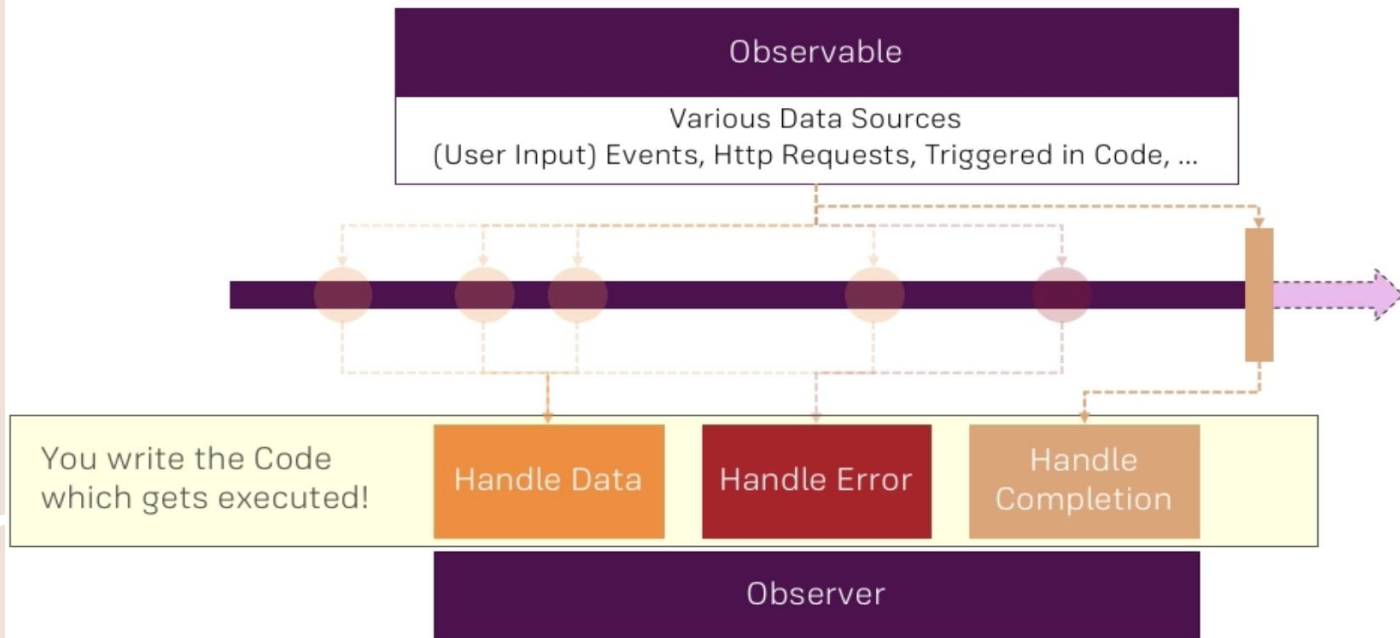
What is Push? In Push systems, the Producer determines when to send data to the Consumer. The Consumer is unaware of when it will receive that data.

Promises are the most common type of Push system in JavaScript today. A Promise (the Producer) delivers a resolved value to registered callbacks (the Consumers), but unlike functions, it is the Promise which is in charge of determining precisely when that value is "pushed" to the callbacks.

RxJS introduces Observables, a new Push system for JavaScript. An Observable is a Producer of multiple values, "pushing" them to Observers (Consumers).

Observable

What is an Observable?

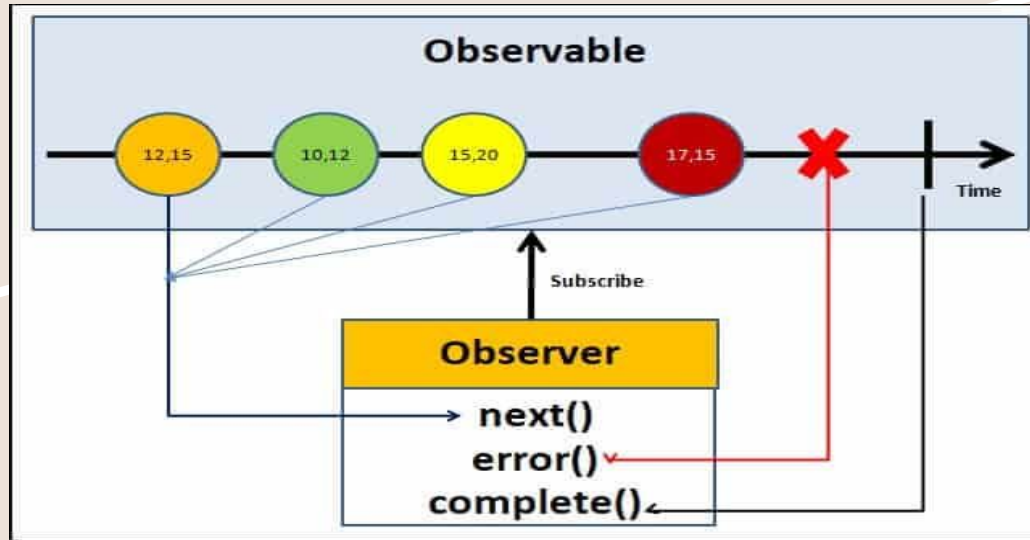


Observables are a push system, like Promises. In fact, they can be “listened to” and they are responsible for emitting values, by wrapping a specific data source .

Observables are used to watch the data streams and emit functions when a value, error or completed signal is returned.

Observables can be **subscribed** to by an observer.

Observables will constantly watch streams and will update accordingly.



Observables Misconceptions

1. **Observables are like functions with zero arguments, but generalize those to allow multiple values.**
Subscribing to an Observable is analogous to calling a Function.

Consider the following:

```
function foo() {  
  console.log('Hello');  
  return 42;  
}
```

```
const x = foo.call(); // same as foo()  
console.log(x);  
const y = foo.call(); // same as foo()  
console.log(y);
```

We expect to see as output:

```
"Hello"  
42  
"Hello"  
42
```


You can write the same behavior above, but with Observables:

```
1.  import { Observable } from 'rxjs';
2.  const foo = new Observable(subscriber => {
3.    console.log('Hello');
4.    subscriber.next(42);
5.  });
6.
7.  foo.subscribe(x => {
8.    console.log(x);
9.  });
10. foo.subscribe(y => {
11.   console.log(y);
12. });
```

Output:

```
"Hello"
42
"Hello"
42
```

And the output is same.

This happens because both functions and Observables are lazy computations. If you don't call the function, the `console.log('Hello')` won't happen.

2. Observables are able to deliver values either synchronously or asynchronously.

Some people claim that Observables are asynchronous. That is not true. If you surround a function call with logs, like this:

```
console.log('before');  
console.log(foo.call());  
console.log('after');
```

```
"before"  
"Hello"  
42  
"After"
```

And this is the same behavior with Observables:

```
console.log('before');  
foo.subscribe(x => {  
  console.log(x);  
});  
console.log('after');
```

What is the difference between an Observable and a function?

1. **Observables can "return" multiple values over time, something which functions cannot.** You can't do this:

```
function foo() {  
  console.log('Hello');  
  return 42;  
  return 100; // dead code. will never  
  happen  
}
```

Functions can only return one value.

Observables, however, can do this:

```
1.  import { Observable } from 'rxjs';
2.  const foo = new Observable(subscriber => {
3.    console.log('Hello');
4.    subscriber.next(42);
5.    subscriber.next(100); // "return" another value
6.    subscriber.next(200); // "return" yet another
7.  });
8.
9.  console.log('before');
10. foo.subscribe(x => {
11.   console.log(x);
12. });
13. console.log('after');
```

With output:

```
"before"
"Hello"
42
100
200
"after"
300
```

2. You can also "return" values asynchronously:

```
1.  import { Observable } from 'rxjs';
2.
3.  const foo = new Observable(subscriber => {
4.    console.log('Hello');
5.    subscriber.next(42);
6.    setTimeout(() => {
7.      subscriber.next(300); // happens asynchronously
8.    }, 1000);
9.  });
10.
11. console.log('before');
12. foo.subscribe(x => {
13.   console.log(x);
14. });
15. console.log('after');
```

With output:

```
"before"
"Hello"
42
"after"
300
```

Conclusion:

- *func.call() means "give me one value synchronously"*
 - *observable.subscribe() means "give me any amount of values, either synchronously or asynchronously"*
- 

Anatomy of an Observable

Observables are created using `new Observable` or a creation operator, are subscribed to with an Observer, execute to deliver next / error / complete notifications to the Observer, and their execution may be disposed.

Core Observable concerns:

- Creating Observables
- Subscribing to Observables
- Executing the Observable
- Disposing Observables

Creating Observables

The Observable constructor takes one argument: the subscribe function.

The following example creates an Observable to emit the string 'hi' every second to a subscriber.

```
import { Observable } from 'rxjs';

const observable = new Observable(function subscribe(subscriber) {
  const id = setInterval(() => {
    subscriber.next('hi')
  }, 1000);
});
```

Observables can be created with `new Observable`. Most commonly, observables are created using creation functions, like `of`, `from`, `interval`, etc.

In the example above, the subscribe function is the most important piece to describe the Observable. Let's look at what subscribing means.

Subscribing to Observables

The Observable observable in the example can be subscribed to, like this:

```
observable.subscribe(x => console.log(x));
```

Subscribing to an Observable is like calling a function, providing callbacks where the data will be delivered to.

A subscribe call is simply a way to start an "Observable execution" and deliver values or events to an Observer of that execution.

Executing Observables

The code inside `new Observable(function subscribe(subscriber) {...})` represents an "Observable execution", a lazy computation that only happens for each Observer that subscribes.

The execution produces multiple values over time, either synchronously or asynchronously.

There are three types of values an Observable Execution can deliver:

- **"Next"** notification: sends a value such as a Number, a String, an Object, etc.
- **"Error"** notification: sends a JavaScript Error or exception.
- **"Complete"** notification: does not send a value.

In an Observable Execution, zero to infinite Next notifications may be delivered. If either an Error or Complete notification is delivered, then nothing else can be delivered afterwards.

The following is an example of an Observable execution that delivers three Next notifications, then completes:

```
import { Observable } from 'rxjs';

const observable = new Observable(function subscribe(subscriber) {
  subscriber.next(1);
  subscriber.next(2);
  subscriber.next(3);
  subscriber.complete();
});
```

Observables strictly adhere to the Observable Contract, so the following code would not deliver the Next notification 4:

```
import { Observable } from 'rxjs';

const observable = new Observable(function subscribe(subscriber) {
  subscriber.next(1);
  subscriber.next(2);
  subscriber.next(3);
  subscriber.complete();
  subscriber.next(4); // Is not delivered because it would violate
the contract
});
```

It is a good idea to wrap any code in subscribe with try/catch block that will deliver an Error notification if it catches an exception:

```
1.  import { Observable } from 'rxjs';  
2.  const observable = new Observable(function subscribe(subscriber) {  
3.    try {  
4.      subscriber.next(1);  
5.      subscriber.next(2);  
6.      subscriber.next(3);  
7.      subscriber.complete();  
8.    } catch (err) {  
9.      subscriber.error(err); // delivers an error if it caught one  
10.    }  
11.  });
```

Disposing Observable Executions

When `observable.subscribe` is called, the Observer gets attached to the newly created Observable execution. This call also returns an object, the Subscription:

```
const subscription = observable.subscribe(x => console.log(x));
```

With `subscription.unsubscribe()` you can cancel the ongoing execution:

```
import { from } from 'rxjs';
```

```
const observable = from([10, 20, 30]);
```

```
const subscription = observable.subscribe(x => console.log(x));
```

```
// Later:
```

```
subscription.unsubscribe();
```

When you subscribe, you get back a Subscription, which represents the ongoing execution. Just call `unsubscribe()` to cancel the execution.

Observer

An Observer is a consumer of values delivered by an Observable. Observers are simply a set of callbacks, one for each type of notification delivered by the Observable: next, error, and complete. The following is an example of a typical Observer object:

```
const observer = {  
  next: x => console.log('Observer got a next value: ' + x),  
  error: err => console.error('Observer got an error: ' + err),  
  complete: () => console.log('Observer got a complete notification'),  
  
};
```

To use the Observer, provide it to the subscribe of an Observable:

```
observable.subscribe(observer);
```

Observers are just objects with three callbacks, one for each type of notification that an Observable may deliver.

Observers in RxJS may also be partial. If you don't provide one of the callbacks, the execution of the Observable will still happen normally, except some types of notifications will be ignored, because they don't have a corresponding callback in the Observer.

Internally in `observable.subscribe`, it will create an Observer object using the first callback argument as the next handler. All three types of callbacks may be provided as arguments:

```
observable.subscribe(  
  x => console.log('Observer got a next value: ' + x),  
  err => console.error('Observer got an error: ' + err),  
  () => console.log('Observer got a complete notification')  
);
```


Operators

RxJS is mostly useful for its operators, even though the Observable is the foundation.

What are operators?

Operators are functions. There are two kinds of operators:

1. **Pipeable Operators:** A Pipeable Operator is a function that takes an Observable as its input and returns another Observable. It is a pure operation: the previous Observable stays unmodified.
2. **Creation Operators:** Creation Operators are the other kind of operator, which can be called as standalone functions to create a new Observable.

An example :

```
import { of } from 'rxjs';
import { map } from 'rxjs/operators';

map(x => x * x)(of(1, 2, 3)).subscribe((v) =>
  console.log(`value: ${v}`));

// Logs:
// value: 1
// value: 4
// value: 9
```

Here, we are using both a creational operator `of()` and a pipeable operator `map()`. The `of()` operator creates a new observable that emits 3 values(1, 2, 3) and then we subscribe to that observable.

Then we are using pipeable operator `map()` that works similar to array `map()` method. It maps all the emitted values as defined inside the function.

Another useful operator is first:

```
first() (of (1, 2, 3)).subscribe((v) => console.log(`value: ${v}`));  
  
// value: 1
```

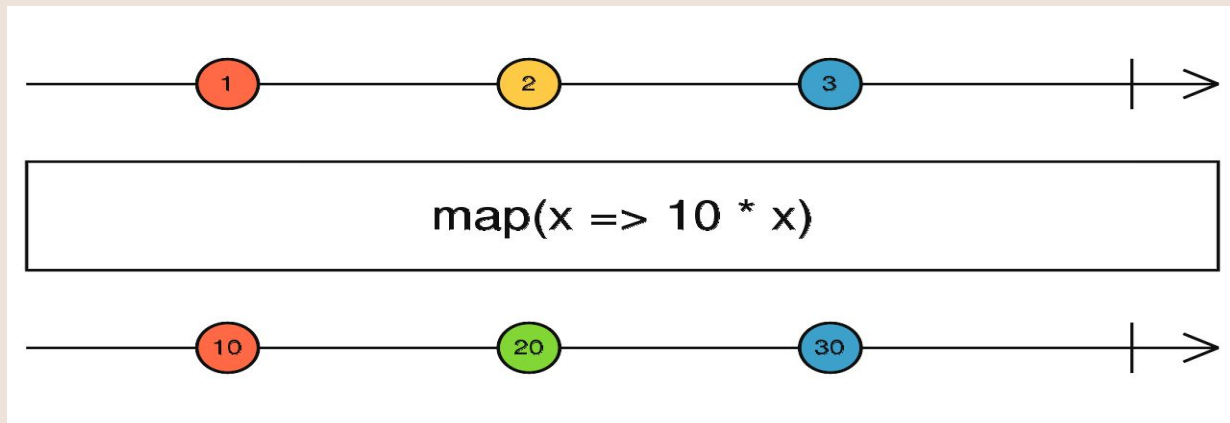
Piping:

Pipeable operators are functions, so they could be used like ordinary functions: `op()(obs)` — but in practice, there tend to be many of them convolved together, and quickly become unreadable: `op4()(op3()(op2()(op1()(obs))))`. For that reason, Observables have a method called `.pipe()` that accomplishes the same thing while being much easier to read:

```
obs.pipe(  
  op1(),  
  op2(),  
  op3()  
)
```

As a stylistic matter, `op()(obs)` is never used, even if there is only one operator; `obs.pipe(op())` is universally preferred.

Map operator example



```
import { fromEvent } from 'rxjs';  
import { map } from 'rxjs/operators';  
  
const clicks = fromEvent(document, 'click');  
const positions = clicks.pipe(map(ev => ev.clientX));  
positions.subscribe(x => console.log(x));
```

Mergemap operator example

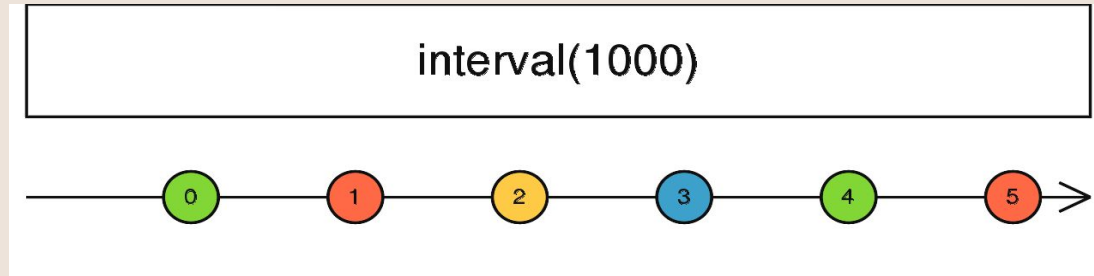
```
1. import { of, interval } from 'rxjs';
2. import { mergeMap, map } from
   'rxjs/operators';
3.
4. const letters = of('a', 'b', 'c');
5. const result = letters.pipe(
6.   mergeMap(x =>
7.     interval(1000).pipe(map(i => x+i))),
8.   );
9. result.subscribe(x =>
10.  console.log(x));
```

```
1. // Results in the
   following:
2. // a0
3. // b0
4. // c0
5. // a1
6. // b1
7. // c1
8. // continues to list
   a,b,c with respective
   ascending integers
```

Map and flatten each letter to an Observable ticking every 1 second.

Let's break the code and understand:

1. Interval: Creates an Observable that emits sequential numbers every specified interval of time.

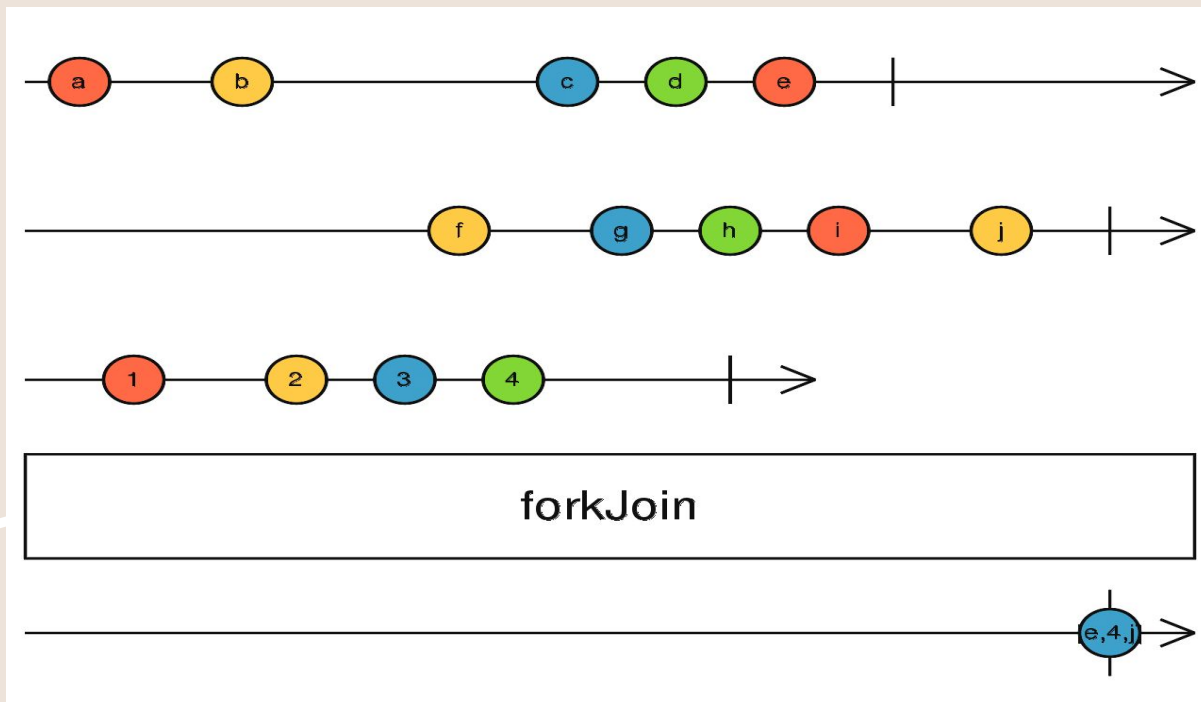


2. Of : Creates an observable that emits the value of sequence provided.(a, b, c)
3. Now, we want to map both the observables and merge them.
4. This task is done by `mergemap()`. It takes in an outer observable that is created using of operator in this case and an inner observable that is created using interval operator.

It maps the value of both the observables and merges them together.

forkJoin() operator

This operator will take in an array or dict object as an input and will wait for the observable to complete and return the last values emitted from the given observable.



Code Example:

```
import { of, forkJoin } from 'rxjs';
```

```
let list1 = of(2, 3, 4, 5, 6);
```

```
let list2 = of(4, 9, 16, 25, 36)
```

```
let final_val = forkJoin([list1, list2]);
```

```
final_val.subscribe(x => console.log(x));
```

Output:

```
[6,36]
```


Subscription

A Subscription is an object that represents a disposable resource, usually the execution of an Observable.

```
import { interval } from 'rxjs';

const observable = interval(1000); //Creates an observable
const subscription = observable.subscribe(x => console.log(x));
// Later:
// This cancels the ongoing Observable execution which
// was started by calling subscribe with an Observer.
subscription.unsubscribe();
```

A Subscription essentially just has an unsubscribe() function to release resources or cancel Observable executions.

Async programming in Js

- JavaScript is a single-threaded programming language which means only one thing can happen at a time. .
- While requesting some data from an API, server might take some time to process the request while blocking the main thread making the web page unresponsive.
- That's where asynchronous programming comes into play.
- Using asynchronous JavaScript (such as callbacks, promises, and `async/await`), you can perform long network requests without blocking the main thread.

Example of asynchronous programming

```
function now() {
```

```
  return 21;
```

```
}
```

```
function later() {
```

```
  answer = answer * 2;
```

```
  console.log("later() execution: ", answer);
```

```
}
```

```
var answer = now();
```

```
setTimeout(later, 1000); // later() execution: 42
```

```
console.log("hey taruna");
```

Output :

hey taruna

later() execution: 42

Event loop in JS

- The Event Loop has one simple job — to monitor the Call Stack and the Callback Queue.
- If Callback queue has codes to execute then it pops the message from it to the Main Stack for the execution.
- Explanation : Once the function hits the return statement, that function is popped from the stack and move to the next stack entry. Meanwhile, Web API is doing its job in the background and remembers what callback function is associated with that job.

Once the job is done, Web API binds the result of that job to the callback function and publishes a message to callback queue with that callback function.

Callback Queue illustration

loupe help

1 Save + Run

```
2  
3 function printHello() {  
4   console.log('Hello from baz');  
5 }  
6  
7 function baz() {  
8   setTimeout(printHello, 3000);  
9 }  
10  
11 function bar() {  
12   baz();  
13 }  
14  
15 function foo() {  
16   bar();  
17 }  
18  
19 foo();
```

Click me! Edit

Call Stack

Web Apis

Callback Queue

The diagram illustrates the execution flow of the provided JavaScript code. The code defines three functions: `printHello()`, `baz()`, and `bar()`. `foo()` calls `bar()`, which calls `baz()`. `baz()` sets a timeout for `printHello()` to be called 3000ms later. The right side of the interface shows the Call Stack, Web Apis, and Callback Queue. A red circular arrow icon indicates the flow of execution from the Callback Queue back to the Call Stack.

Callback and Microtask Queue illustration



Event loop summed up:

```
console.log('script start');

//callback queue
setTimeout(function () {
  console.log('setTimeout');
}, 0);

//microtask queue
Promise.resolve()
  .then(function () {
    console.log('promise1');
  })
  .then(function () {
    console.log('promise2');
  });

console.log('script end');
```

Output:
script start
script end
Promise1
Promise2
setTimeout

References:

[https://christianlydemann.com/are-observables-async/#:~:text=A%20lot%20of%20\(even%20experienced,code%20async%20in%20the%20browser.](https://christianlydemann.com/are-observables-async/#:~:text=A%20lot%20of%20(even%20experienced,code%20async%20in%20the%20browser.)

<https://luukgruijs.medium.com/understanding-rxjs-map-mergemap-switchmap-and-concatmap-833fc1fb09ff>

THANK YOU !!